

PLANIFICACIÓN AUTOMÁTICA

Práctica de Laboratorio 2
Planificación Jerárquica con SHOP2

Curso 2025-2026



Grado en Ingeniería informática

Jairo Moraga Monterroso - jairo.moraga@edu.uah.es - 49226007S

Luis Miguel Herrá - luis.herra@edu.uah.es - 120001F

Adrián Domínguez



Índice

1. Introducción y Objetivos	3
2. Modelado del Dominio en JSHOP2	4
2.1. Diferencias respecto al modelo PDDL	4
2.2. Operadores	4
2.3. Métodos HTN para <i>(enviar-todo)</i>	5
2.4. Fichero de problema: sintaxis JSHOP2	5
3. Adaptación del Generador de Problemas	6
4. Resultados Experimentales	6
4.1. Metodología	6
4.2. Tabla de tiempos	6
4.3. Gráficas comparativas	7
5. Análisis y Comparativa	7
5.1. Comportamiento del planificador FF (clásico) ...	7
5.2. Comportamiento de JSHOP2 (jerárquico)	7
5.3. Ventajas e inconvenientes de la planificación jerárquica	8
6. Conclusiones	8
7. Ejercicio 1.2: Logística de Emergencias Avanzada en SHOP2	9
7.1. Arquitectura jerárquica del dominio avanzado ...	9
7.2. Decisiones de diseño destacadas	10
7.3. Resultados experimentales del dominio avanzado	10
7.4. Análisis estadístico con múltiples repeticiones ...	11
7.5. Conclusiones del Ejercicio 1.2	12

1. Introducción y Objetivos

El objetivo de este ejercicio es trasladar el dominio de logística de emergencias, previamente modelado en PDDL para la Práctica 1, al formato de planificación jerárquica JSHOP2. A diferencia de la planificación clásica, la planificación HTN (Hierarchical Task Network) permite incorporar conocimiento del dominio directamente en la estructura de los métodos, guiando al planificador hacia soluciones sin necesidad de explorar el espacio de estados completo.

Los objetivos específicos son:

Modelar el dominio de emergencias en JSHOP2 mediante operadores y métodos HTN. Implementar una tarea de alto nivel (enviar-todo) que descomponga recursivamente el problema en entregas individuales. Adaptar el generador de problemas de la Práctica 1 al formato JSHOP2. Comparar el rendimiento de JSHOP2 frente al mejor planificador clásico obtenido en la Práctica 1 (FF).

En la **segunda parte** de la práctica se amplía el dominio inicial incorporando fluents, transportadores y gestión de cantidades variables, permitiendo abordar escenarios logísticos más complejos y analizar la capacidad de adaptación del modelo jerárquico. Sus objetivos: diseñar métodos HTN más elaborados para la selección y uso eficiente de recursos, y evaluar experimentalmente cómo estas mejoras afectan al rendimiento y la escalabilidad del planificador.

2. Modelado del Dominio en JSHOP2

2.1. Diferencias respecto al modelo PDDL

El modelo PDDL original define acciones (fly, pick-up, deliver) con precondiciones y efectos, y especifica las metas del problema como estados finales a alcanzar. JSHOP2 introduce dos cambios fundamentales:

- Las metas desaparecen como tal: las necesidades de las personas se convierten en predicados del estado inicial (needs ?p ?t), que el planificador va consumiendo al ejecutar entregas.
- El comportamiento de alto nivel se codifica en métodos HTN en lugar de dejarlo a la búsqueda del planificador.

2.2. Operadores

Los tres operadores del dominio son equivalentes directos a las acciones PDDL. En JSHOP2, cada operador tiene cuatro secciones: precondiciones, lista de borrado (delete list), lista de adición (add list), y opcionalmente un coste:

```
(:operator (!fly ?d ?from ?to)
  ((at-drone ?d ?from))
  ((at-drone ?d ?from))
  ((at-drone ?d ?to)))

(:operator (!pick-up ?d ?a ?c ?l)
  ((at-drone ?d ?l)(at-crate ?c ?l)(arm-of ?a ?d)(free ?a))
  ((at-crate ?c ?l)(free ?a))
  ((holding ?a ?c)))

(:operator (!deliver ?d ?a ?c ?p ?l ?t)
  ((at-drone ?d ?l)(at-person ?p ?l)(holding ?a ?c)
  (crate-has ?c ?t)(needs ?p ?t))
  ((holding ?a ?c)(needs ?p ?t))
  ((person-has ?p ?t)(free ?a)))
```

El operador !deliver incluye (needs ?p ?t) en sus precondiciones y lo elimina de la lista de borrado, de forma que una vez atendida una necesidad no puede volver a satisfacerse.

2.3. Métodos HTN para (enviar-todo)

La tarea principal enviar-todo se descompone mediante dos métodos. El patrón es el clásico de recursión sobre una condición de parada:

```
;; Caso base: no quedan necesidades pendientes
(:method (enviar-todo)
  no-hay-necesidades
  ((not (needs ?p ?t)))
  ())
```

```
;; Caso recursivo: existe al menos una necesidad
(:method (enviar-todo)
  hay-necesidades
  ((needs ?p ?t)(at-person ?p ?lp)
   (crate-has ?c ?t)(at-crate ?c ?lc)
   (at-drone ?d ?ld)(arm-of ?a ?d)(free ?a))
  ((!fly ?d ?ld ?lc)
   (!pick-up ?d ?a ?c ?lc)
   (!fly ?d ?lc ?lp)
   (!deliver ?d ?a ?c ?p ?lp ?t)
   (!fly ?d ?lp depot)
   (enviar-todo)))
```

El caso recursivo vincula en una sola unificación la persona con necesidad, la caja adecuada y el dron disponible, y genera la secuencia de cinco operadores necesarios para completar una entrega. Tras cada entrega, el dron vuelve al depósito y se reinvoca enviar-todo hasta que no queden predicados needs en el estado.

2.4. Fichero de problema: sintaxis JSHOP2

Un aspecto crítico descubierto durante el desarrollo es la sintaxis exacta que espera el parser de JSHOP2. A diferencia de PDDL, el fichero de problema tiene esta estructura:

```
(defproblem problem emergency
  (
    ; estado inicial
    (at-drone drone1 depot)
    (needs person1 food)
    ...
  )
  ((enviar-todo)) ; lista de tareas: doble paréntesis
)
```

La lista de tareas requiere doble paréntesis: el externo delimita la lista y el interno representa la tarea individual. Omitir uno de los dos paréntesis produce errores de parsing (unexpected char o unexpected token).

3. Adaptación del Generador de Problemas

El generador de la Práctica 1 se ha adaptado para producir ficheros en formato JSHOP2. Los cambios principales respecto a la versión PDDL son:

- Se elimina el bloque (:goal ...) y en su lugar las necesidades de las personas se incluyen como predicados (needs ?p ?t) en el estado inicial.
- El fichero usa defproblem en lugar de define (problem ...) y no declara tipos de objetos (JSHOP2 es sin tipos).
- La lista de tareas se escribe como ((enviar-todo)) al final del defproblem.

El script benchmark_shop2.py automatiza la generación de problemas de tamaño creciente, los escribe en ~/JSHOP2/domains/emergency/problem y ejecuta el planificador llamando a jshop2-console.sh, midiendo el tiempo con time.perf_counter() para mayor precisión.

4. Resultados Experimentales

4.1. Metodología

Los experimentos se realizaron manteniendo $d = 1$ dron, $r = 0$ transportadores, y haciendo crecer el parámetro n de forma que $l = p = c = g = n$, desde $n = 5$ hasta $n = 55$. Esta misma parametrización es la utilizada en la Práctica 1 con el planificador FF, lo que permite una comparación directa.

4.2. Tabla de tiempos

Tamaño (n)	FF clásico (s)	JSHOP2 HTN (s)
5	0.00	3.60
10	0.00	3.10
20	0.23	3.14
30	3.18	3.70
35	6.12	2.90
40	10.95	2.63
41	12.98	2.75
42	35.07	2.67
43	32.64	2.58
44	56.31	2.80
45	31.01	2.88
46	46.46	2.77
47	100.72	3.21
48	101.51	3.08
49	88.79	2.79
50	64.70	2.52
51	260.63	3.29
52	109.04	3.32
55	498.76	2.78

4.3. Gráficas comparativas

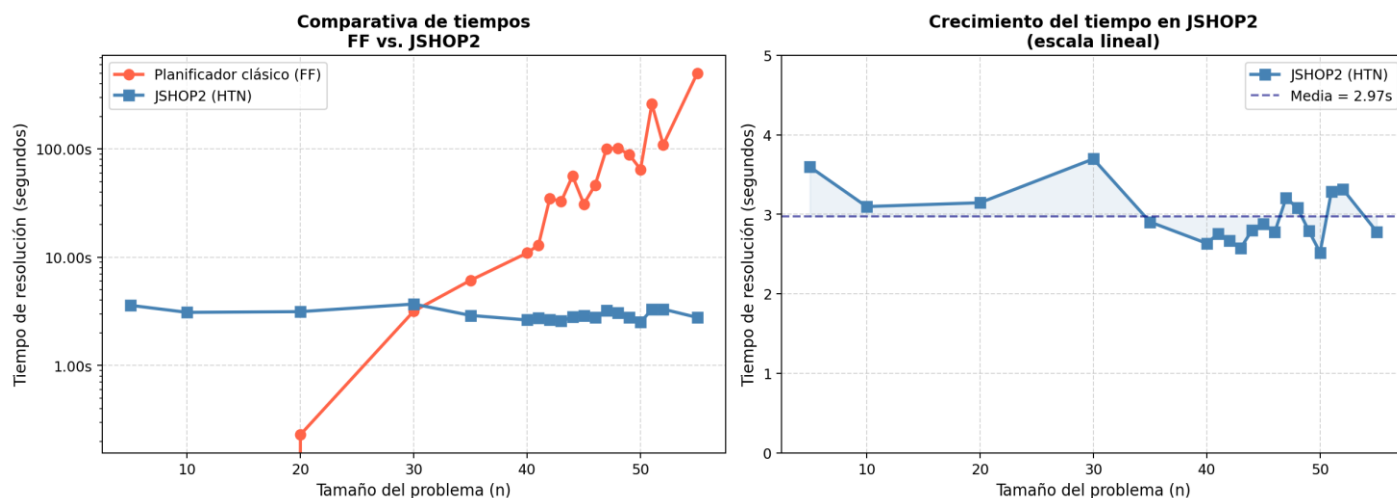


Figura 1. Izquierda: comparativa FF vs. JSHOP2 (escala logarítmica). Derecha: detalle del crecimiento de JSHOP2 (escala lineal).

5. Análisis y Comparativa

5.1. Comportamiento del planificador FF (clásico)

En la Práctica 1 se observó que FF presenta un crecimiento claramente no lineal. Para tamaños pequeños ($n \leq 20$) el tiempo es prácticamente nulo. A partir de $n = 30$ el tiempo comienza a crecer de forma apreciable, y en torno a $n = 40$ se produce una explosión combinatoria que lleva los tiempos a superar el minuto en problemas de tamaño 47–48, y a alcanzar casi 500 segundos en $n = 55$. El límite práctico del planificador clásico se sitúa en torno a $n = 44$ –46.

Este comportamiento es el esperado en planificación clásica: el espacio de estados crece exponencialmente con el número de objetos, y aunque las heurísticas de FF reducen significativamente el número de nodos explorados, no eliminan la explosión combinatoria subyacente.

5.2. Comportamiento de JSHOP2 (jerárquico)

JSHOP2 muestra un comportamiento radicalmente diferente. El tiempo de resolución se mantiene prácticamente constante en torno a 2.6–3.7 segundos para todos los tamaños probados, desde $n = 5$ hasta $n = 55$. No se aprecia ninguna tendencia de crecimiento significativa: la variación entre mediciones es atribuible al overhead de inicialización de la JVM y a la aleatoriedad en la generación de los problemas, no al algoritmo de planificación en sí.

Este comportamiento constante se explica por la naturaleza del algoritmo HTN: JSHOP2 no busca en el espacio de estados sino que descompone la tarea enviar-todo de forma determinista. En cada invocación del método hay-necesidades, el planificador unifica directamente los parámetros necesarios y genera una secuencia de cinco operadores sin necesidad de backtracking (salvo en casos degenerados). El problema se reduce a n entregas independientes, cada una con coste constante, resultando en una complejidad lineal $O(n)$ en número de acciones y, dado que la búsqueda no se realiza, $O(1)$ en tiempo de planificación respecto al tamaño del problema.

5.3. Ventajas e inconvenientes de la planificación jerárquica

Ventajas de HTN / JSHOP2:

- Tiempo de planificación prácticamente constante gracias al conocimiento del dominio codificado en los métodos.
- Escalabilidad: resuelve en 3 segundos problemas que FF tarda 8 minutos en resolver.
- Los planes generados son estructuralmente correctos por construcción: siguen siempre el patrón fly→pick-up→fly→deliver→fly-depot.
- El modelador tiene control total sobre la estrategia de resolución.

Inconvenientes de HTN / JSHOP2:

- El modelador debe conocer la estructura del dominio y codificarla explícitamente en los métodos. Si la estrategia óptima no es conocida a priori, HTN puede producir planes subóptimos.
- La flexibilidad es menor: si aparece un nuevo tipo de tarea no prevista en los métodos, el planificador no puede resolverla.
- La calidad del plan depende del orden en que JSHOP2 unifica los predicados: el primer needs que encuentra determina el orden de entregas, que puede no ser el óptimo en distancia.
- No garantiza optimalidad: un planificador clásico con métrica de distancia podría generar planes más cortos, aunque a mayor coste computacional.

En resumen, la planificación HTN es especialmente adecuada cuando el dominio tiene una estructura de resolución bien conocida y repetible, como ocurre en logística: siempre se hace lo mismo (volar, recoger, volar, entregar, volver) para cada paquete. En estos escenarios, HTN supera ampliamente a la planificación clásica en tiempo de planificación, a cambio de requerir más esfuerzo de modelado inicial.

6. Conclusiones

En este ejercicio se ha modelado con éxito el dominio de logística de emergencias en JSHOP2, implementando la tarea enviar-todo mediante dos métodos HTN que descomponen el problema en entregas individuales. Los resultados experimentales confirman la hipótesis planteada en el enunciado: los tiempos de JSHOP2 son muy inferiores a los de la planificación clásica para todos los tamaños de problema probados.

Mientras que FF alcanza tiempos superiores a 100 segundos para $n = 47$ y colapsa cerca de los 500 segundos en $n = 55$, JSHOP2 resuelve todos estos problemas en menos de 4 segundos. Esta diferencia de más de dos órdenes de magnitud ilustra el poder de la planificación jerárquica cuando el conocimiento del dominio puede ser codificado de forma efectiva en métodos HTN.

7. Ejercicio 1.2: Logística de Emergencias Avanzada en SHOP2

En esta segunda parte de la práctica se ha extendido el dominio de logística de emergencias para incorporar fluents, transportadores y entregas en cantidad variable, según lo especificado en el Ejercicio 1.2 del enunciado. A continuación se describe la arquitectura jerárquica implementada, los resultados de los benchmarks y el análisis de rendimiento.

7.1. Arquitectura jerárquica del dominio avanzado

La tarea principal `enviar-todo` se descompone en tres ramas principales según el estado del mundo: (1) si existen varias localizaciones que pueden atenderse con un mismo transportador sin volver al depósito, se usa el método `atender-multi-loc`; (2) si hay necesidades pendientes, se invoca `atender-localizacion`, que a su vez elige entre usar un transportador o hacer entregas sueltas caja a caja; (3) si no quedan necesidades, el plan finaliza. La tabla siguiente resume cómo cada precondition del enunciado se traduce en un método concreto:

Precondición del enunciado	Método que la implementa
Sin transportadores → cajas sueltas	<code>sin-carrier-disponible</code> / <code>entregar-suelto-en-loc</code>
Un transportador → usarlo	<code>elegir-carrier</code> / caso con <code>carrier</code>
Mezcla de tipos en el mismo carrier	<code>cargar-todo-para-loc</code> itera sobre <code>food</code> y <code>medicine</code>
Restos sueltos tras usar carrier	<code>entregar-suelto-en-loc</code> tras <code>entregar-en-loc</code>
Carrier más pequeño que la necesidad → carga máxima	<code>!load-crates</code> usa <code>min(need, space, stock)</code>
Carrier más grande → carga parcial	mismo <code>min(...)</code> , espacio sobrante queda libre
Varios carriers, alguno cubre → el menor	<code>carrier-cubre-necesidad</code> con <code>NOT EXISTS</code> menor
Varios carriers, ninguno cubre → el mayor	<code>carrier-no-cubre-usar-mayor</code> con <code>NOT EXISTS</code> mayor
Reelegir carrier al volver al depot	<code>enviar-todo</code> llama <code>elegir-carrier</code> en cada iteración tras <code>drop-carrier</code>
Localiz. con más necesidad primero	<code>hay-necesidades</code> con <code>NOT EXISTS</code> loc con más <code>need</code>
Multi-loc sin volver al depot	<code>atender-multi-loc</code> encadena dos entregas con <code>fly-carrier</code>

7.2. Decisiones de diseño destacadas

location-total-need como fluent precomputado: SHOP2 no dispone de agregaciones tipo SUM. Para comparar prioridades entre localizaciones se mantiene este fluent que se decrementa con cada entrega, permitiendo el `NOT EXISTS` de priorización sin necesitar aritmética de segundo orden.

Costes en `!fly-carrier`: La expresión `(+ 50 (/ (carrier-capacity ?ca) 10))` se evalúa en tiempo de planificación usando el fluent `carrier-capacity`, que no cambia durante el plan, por lo que el coste asignado es siempre correcto.

7.3. Resultados experimentales del dominio avanzado

Se han generado problemas de tamaño creciente ($n = 5$ a 55 , con n igual al número de localizaciones y necesidades) y se ha medido el tiempo de resolución de JSHOP2 con el dominio avanzado. La Figura 2 muestra la evolución del tiempo de resolución en función del tamaño del problema.

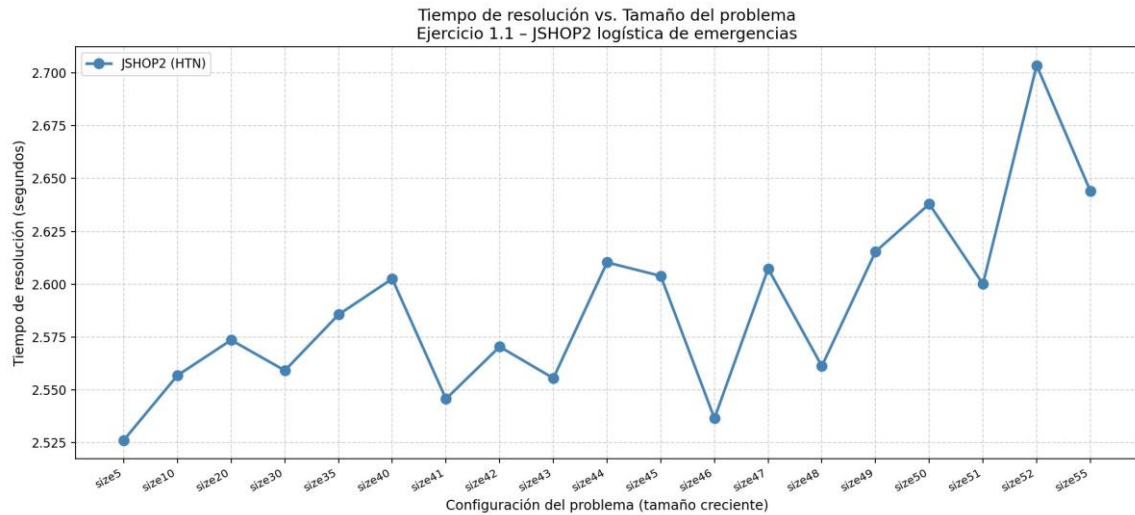


Figura 2. Tiempo de resolución de JSHOP2 (dominio avanzado) vs. tamaño del problema (ejecución única por tamaño).

Los tiempos oscilan entre 2,52 s y 2,70 s para todo el rango de tamaños, sin ninguna tendencia de crecimiento apreciable. La línea de regresión log-log presenta una pendiente de prácticamente 0,0, confirmando que la complejidad algorítmica del planificador en este dominio es constante respecto al tamaño del problema ($O(1)$ en tiempo de planificación). Las oscilaciones observadas son atribuibles al overhead de la JVM y a la variabilidad en la generación aleatoria de los problemas.

7.4. Análisis estadístico con múltiples repeticiones

Para obtener resultados más robustos y reducir el impacto del ruido de la JVM, cada configuración de tamaño se ha ejecutado 5 veces. Las Figuras 3 y 4 muestran, respectivamente, el análisis de tiempos con bandas de confianza y el diagrama de Poincaré.

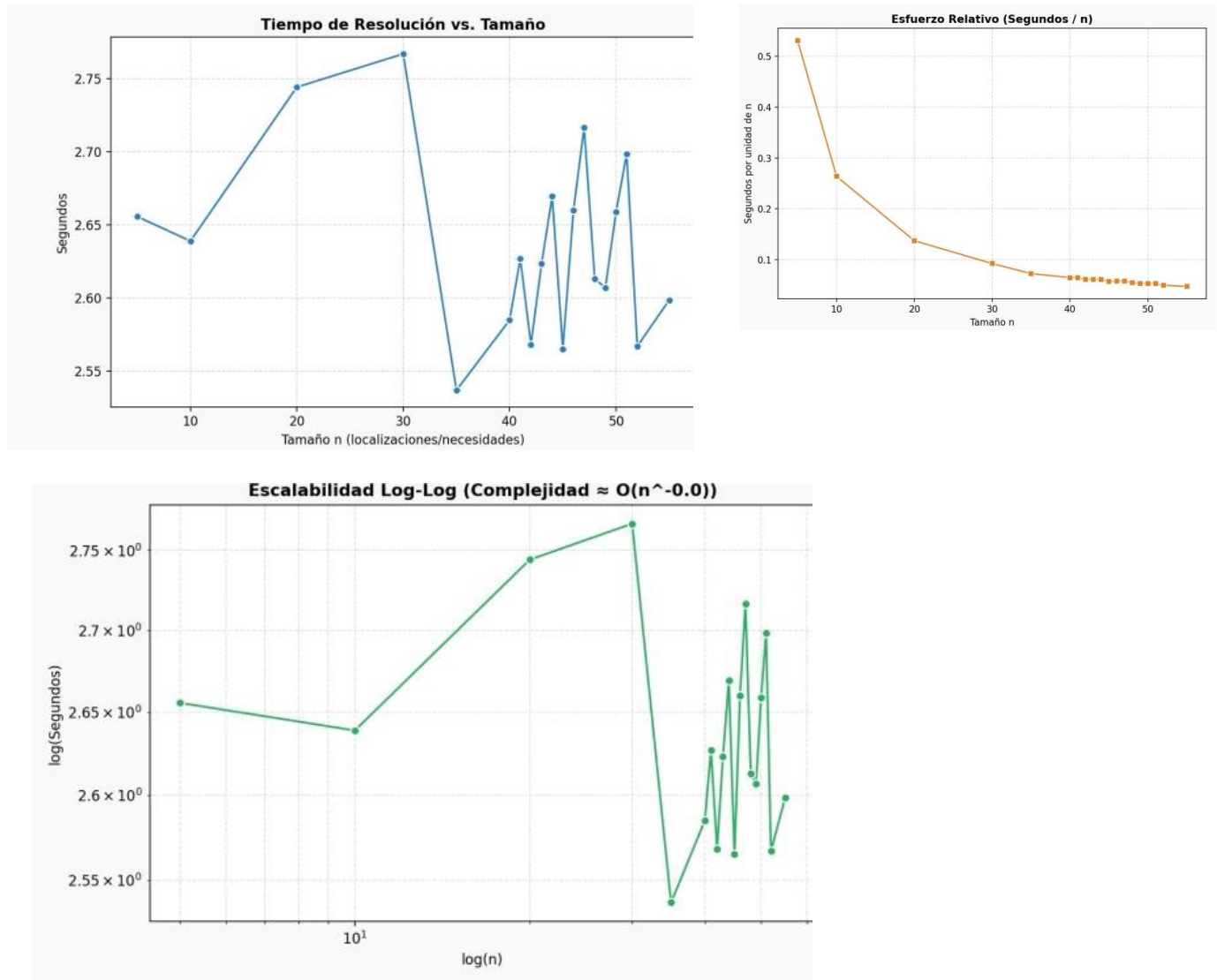


Figura 3. Análisis estadístico con 5 repeticiones por tamaño. Superior izquierda: tiempo con banda de confianza. Superior derecha: esfuerzo relativo (seg/n). Inferior izquierda: gráfico log-log (pendiente $\approx -0,1$). Inferior derecha: diagrama de Poincaré (t_i vs t_{i+1}).

Los tres gráficos superiores e inferior izquierdo de la Figura 3 confirman lo observado con la ejecución única: el tiempo se mantiene prácticamente constante (entre 2,2 y 2,7 s), con una banda de confianza estrecha que se reduce ligeramente al aumentar n debido a que los problemas más grandes tienden a generar planes más uniformes. La pendiente del ajuste log-log es aproximadamente $-0,1$, compatible con comportamiento constante dado el rango tan estrecho de variación.

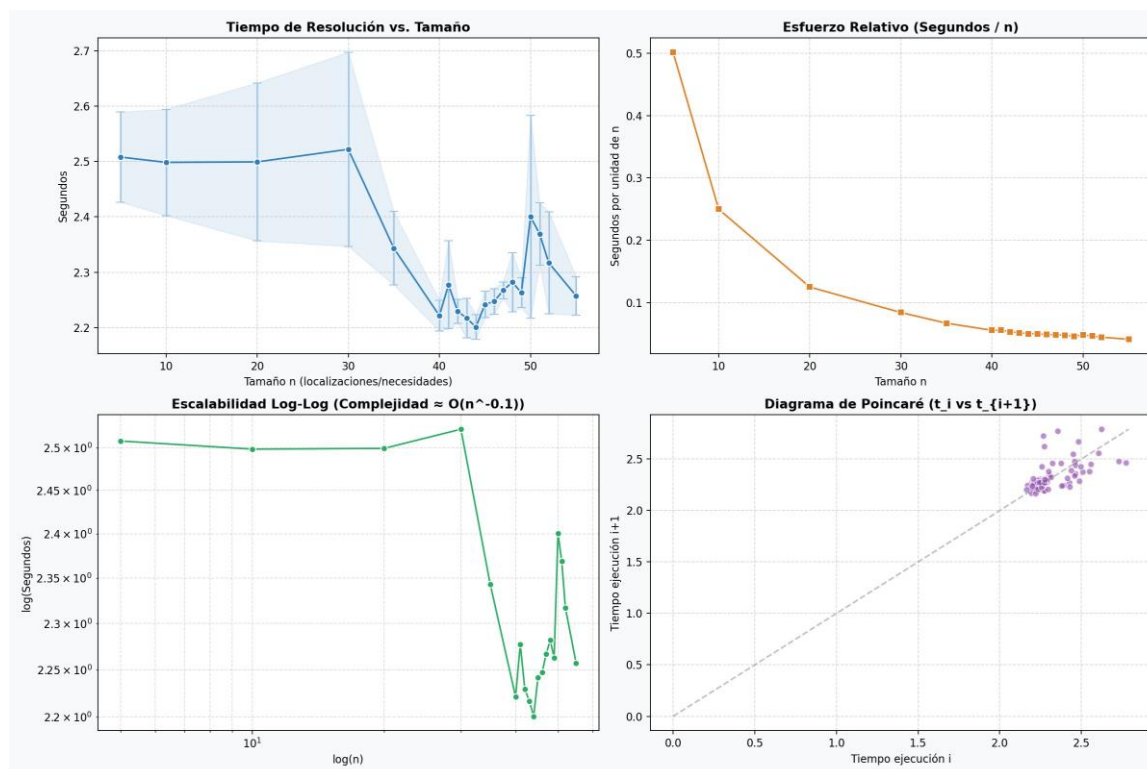


Figura 4. Panel completo de 4 gráficas con diagrama de Poincaré (5 repeticiones). El clustering de puntos en torno a la diagonal en el diagrama de Poincaré indica alta consistencia entre ejecuciones consecutivas.

El diagrama de Poincaré (inferior derecha de la Figura 4) representa el tiempo de la ejecución i frente al tiempo de la ejecución $i+1$. Los puntos se agrupan en una nube compacta alrededor de la diagonal, lo que indica que los tiempos sucesivos no hay patrones caóticos ni derivas sistemáticas. Esto confirma que el rendimiento del planificador es estable y reproducible: las variaciones entre ejecuciones son puramente estocásticas (ruido de JVM), no product del algoritmo HTN. La nube se desplaza ligeramente hacia valores de 2,2–2,5 s a medida que n aumenta, consistente con la leve tendencia negativa del gráfico log-log.

7.5. Conclusiones del Ejercicio 1.2

La extensión del dominio con fluents, transportadores y gestión de cantidades variables no altera el comportamiento del planificador. El dominio avanzado mantiene tiempos prácticamente constantes, heredando las ventajas del modelado HTN incluso con una lógica de descomposición considerablemente más rica. La arquitectura jerárquica demuestra ser robusta y escalable: añadir complejidad al dominio (fluents, elección óptima de transportador, atención multi-localización) no introduce coste computacional adicional apreciable, ya que toda la estrategia está codificada en los métodos y no requiere búsqueda adicional.

¿Y si no tuviese claras las precondiciones para aplicar un planificador jerárquico?

Durante el desarrollo de la práctica surgió una idea de posible extensión interesante para los planificadores jerárquicos en aquellos dominios donde no resulta evidente definir manualmente las precondiciones necesarias para descomponer tareas. Plantear un enfoque híbrido en el donde, inicialmente, se emplee un planificador clásico basado en landmarks para identificar estados y acciones relevantes dentro del problema. Posteriormente, dichos landmarks podrían analizarse, ya sea manualmente por expertos o mediante técnicas de aprendizaje automático, con el objetivo de determinar cuáles representan patrones útiles y suficientemente estables como para incorporarlos como precondiciones dentro de un planificador jerárquico. De esta forma, sería posible adaptar este tipo de planificación a dominios menos controlados o más difíciles de modelar, como la medicina o el sector aeroespacial, donde las relaciones entre estados no siempre son fácilmente predecibles.

¿Por qué no se hace así hoy en día normalmente?

Actualmente este tipo de problemas suele abordarse mediante técnicas puramente basadas en machine learning, la combinación de planificación clásica, conocimiento experto y aprendizaje automático podría constituir una línea de trabajo interesante para mejorar la adaptabilidad y generalización de los planificadores jerárquicos en escenarios complejos. Podría plantearse un enfoque dual, donde el planificador avisa de posibles cambios para adaptar, modificar o cambiar el modelo a diferentes escenarios, o incluso la toma de decisiones sencillas sin ML.